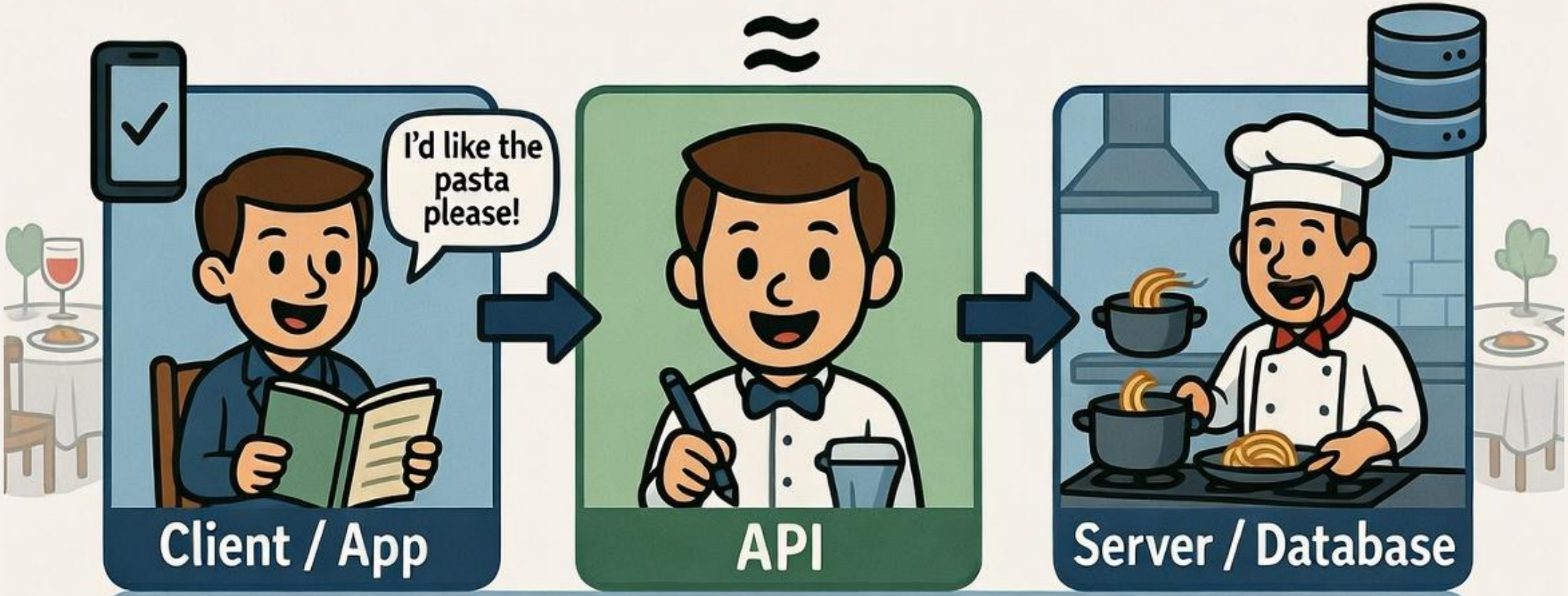
IS487
Emerging Topics in IS

Dr. Abbas Malik
amaalik@psu.edu.sa

THE DIGITAL WAITER

≈



- You (the Client/App) don't go into the kitchen to get your food (the Database).
- You interact with a waiter (the API).
- You tell the waiter what you want, and they bring it back.

API is the intermediary that defines how your app can talk to a server. It takes your request, gets the necessary data from the server, and returns a response.

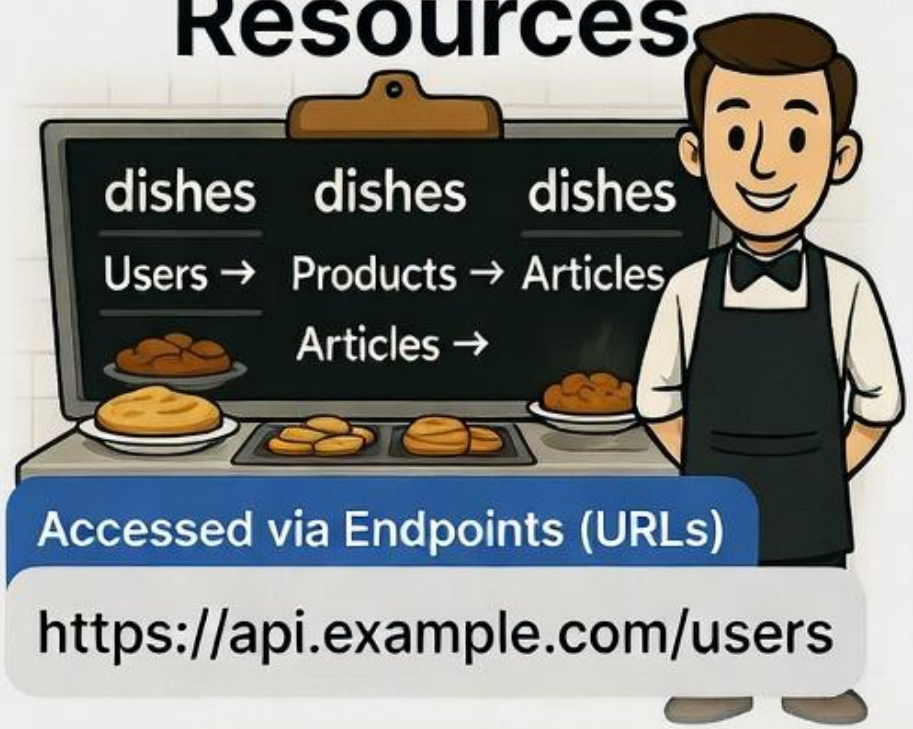
REST - Architectural Guidelines

- Rules for building web APIs
- Not a protocol, but an architectural style
- Leverages standard HTTP methods (verbs) to perform actions on resources (data)

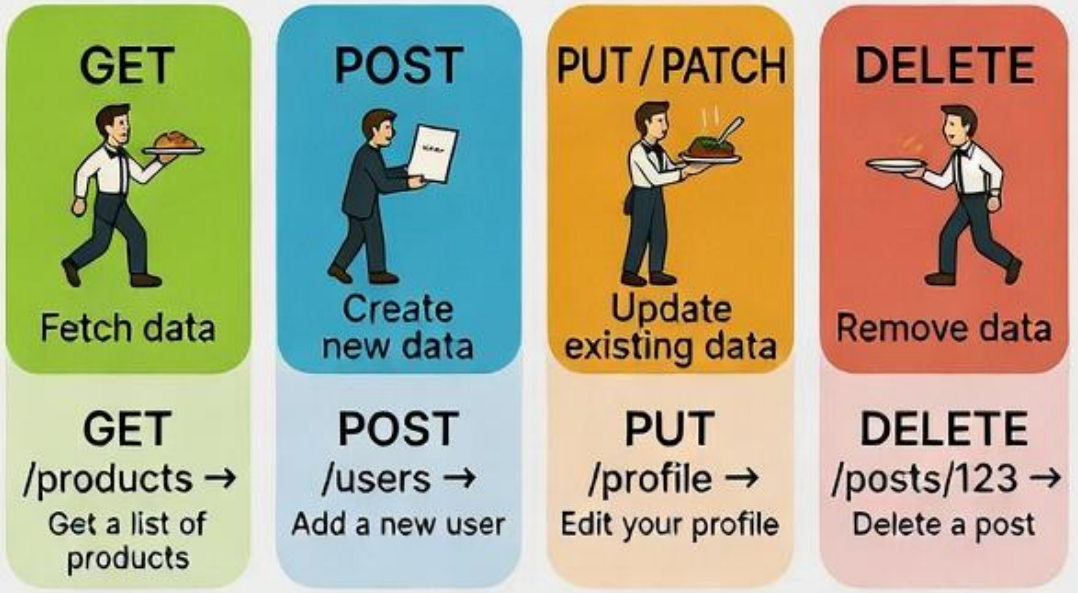
THE DIGITAL WAITER

Resources & HTTP Methods

Resources



HTTP METHODS



Resources are the data you work with. HTTP Methods are the actions the API performs on those resources via Endpoints.

HTTP Status Codes: Server's Response

- Every HTTP response comes with a status code that tells you what happened
- **1xx - Informational**: Request received.
- **2xx - Success**: Request successful. (200 OK, 201 Created)
- **3xx - Redirection**: Further action needed
- **4xx - Client Error**: 400 Bad Request, 404 Not Found
- **5xx - Server Error**: 500 Internal Server Error

THE DIGITAL WAITER
HTTP Status Codes

Every HTTP response comes with a status code that tells you what happened.

1xx

Informational
Request received

2xx

Success
The request was successful
200 OK
201 Created

3xx

Redirection
Further action needed

4xx

Client Error
Problem with the request from the App
400 Bad Request
404 Not Found

5xx

Server Error
The server failed to fulfill a valid request
500 Internal Server Error

Status codes help you understand exactly what happened with your API request.

JSON: Language of Data Exchange

- **JSON (JavaScript Object Notation)** - most common format for sending and receiving data
- Lightweight
- Language-independent
- Easy for both humans and machines to read
- Looks a lot like Dart Map.

THE DIGITAL WAITER

Code Example: JSON vs Dart

From the API

```
{n:
userId: 1,
id: 1,
title: 'Sample Post Title',
body: 'This is the content of the
}}
```

Dart

In Your Flutter App

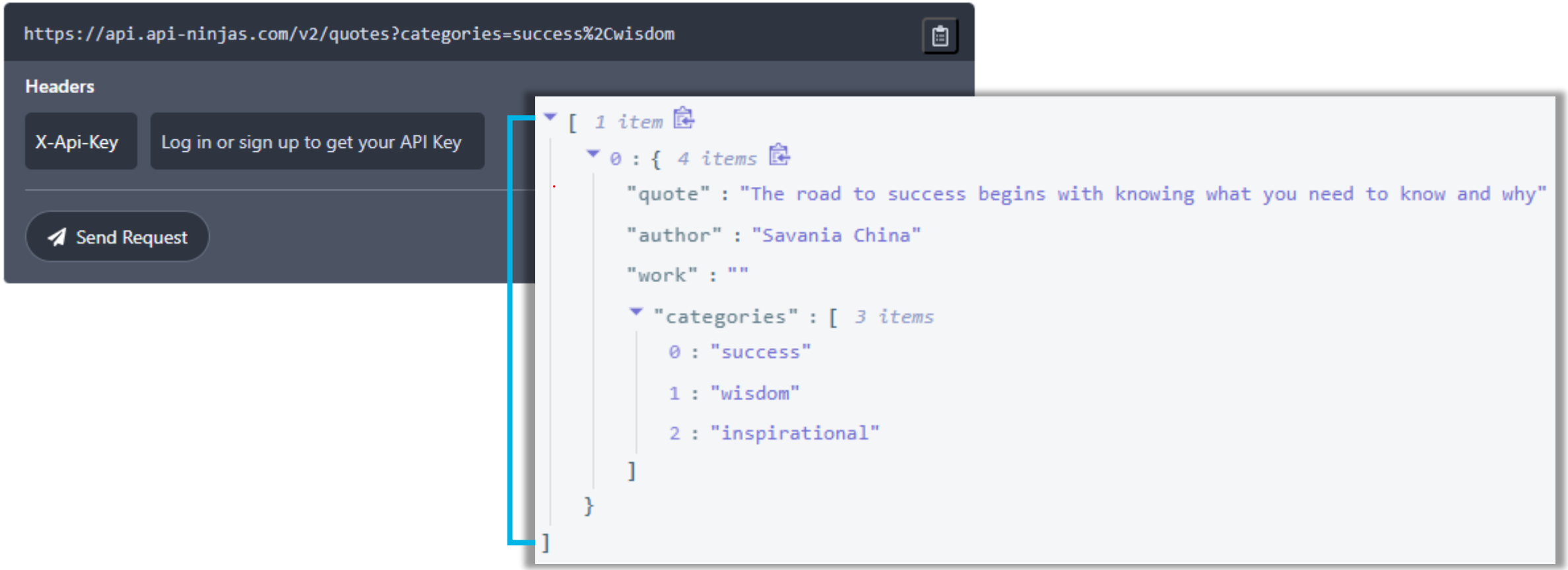
```
Map<String, dynamic> post = {

    'userId': 1,
    'id': 1,
    'title': 'Sample Post Title',
    'body': 'This is the content of
};
```

APIs typically send data in JSON format.

Quotes API

https://api-ninjas.com/api/quotes



Quotes API

- /v2/quotes - Get quotes
- /v2/randomquotes - Get random quotes
- /v2/quoteoftheday - Get today's quote (same quote all day)

Quotes API

https://api.api-ninjas.com/v2/quotes?categories=success%2Cwisdom&limit=4

Parameters

`categories` optional

Comma-separated list of categories to include in results (results will match all of the categories). Example: `categories=wisdom,success`

`exclude_categories` optional

Comma-separated list of categories to exclude from results (results will not match any of the categories). Example: `exclude_categories=love,philosophy`

`author` optional

Filter quotes by author name (partial match supported). Example: `author=Einstein`

`work` optional

Filter quotes by work title (partial match supported). Example: `work=War`

`limit` premium only

Number of results to return. Must be between `1` and `100`. Default is `1`.

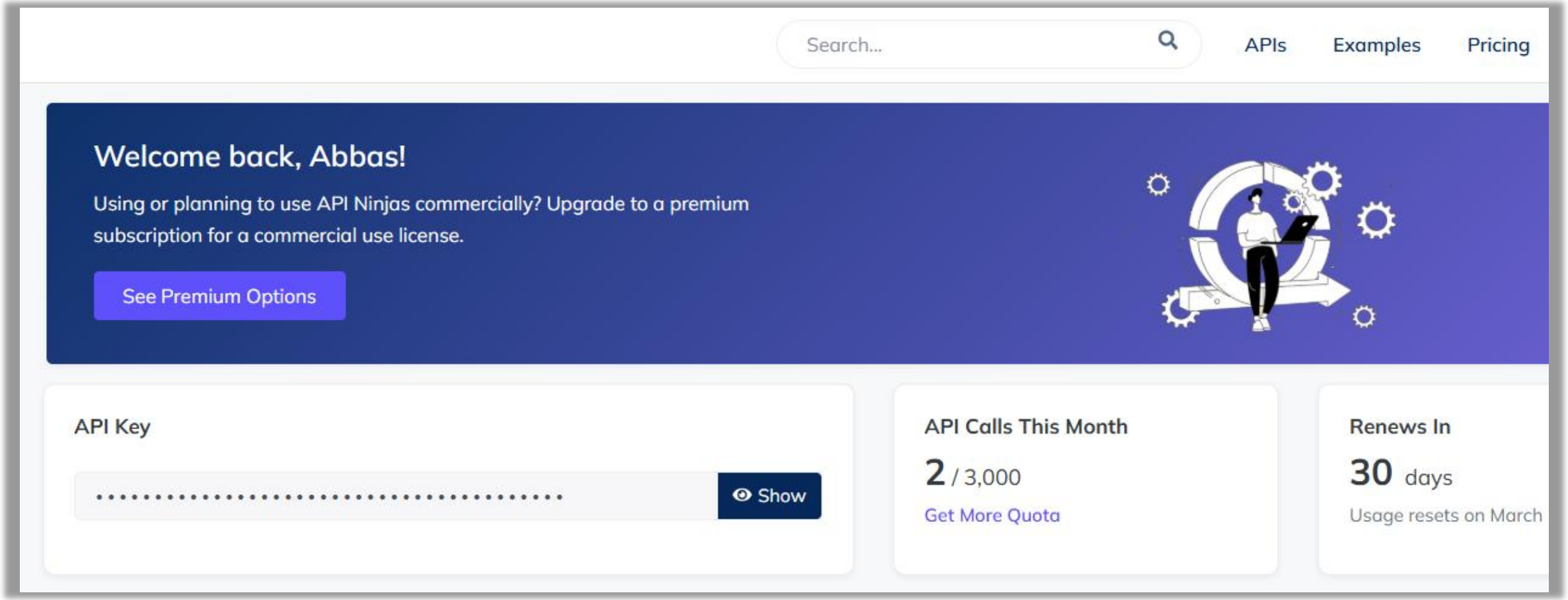
`offset` premium only

Number of results to skip for pagination. Default is `0`.

Quotes API - Categories

- wisdom
- philosophy
- life
- truth
- inspirational
- relationships
- love
- faith
- humor
- success
- courage
- happiness
- art
- writing
- fear
- nature
- time
- freedom
- death
- leadership

Get Yout API Key



Data Model in App

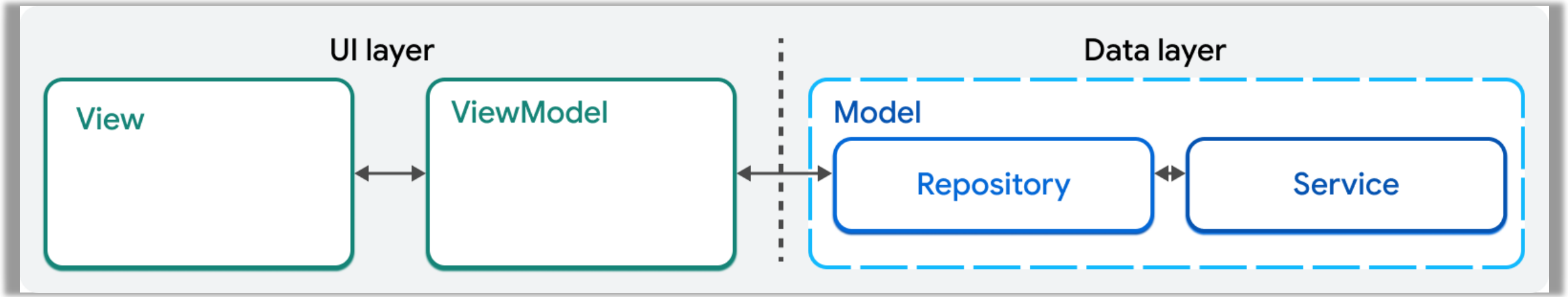
```
{
    "quote":"The road to success begins with
    knowing what you need to know and why"
    "author":"Savania China"
    "work":""
    "categories":[
        0:"success"
        1:"wisdom"
        2:"inspirational"
    ]
}
```

```
class Quote {
  final String quote;
  final String author;
  final List<String> categories;
}
```

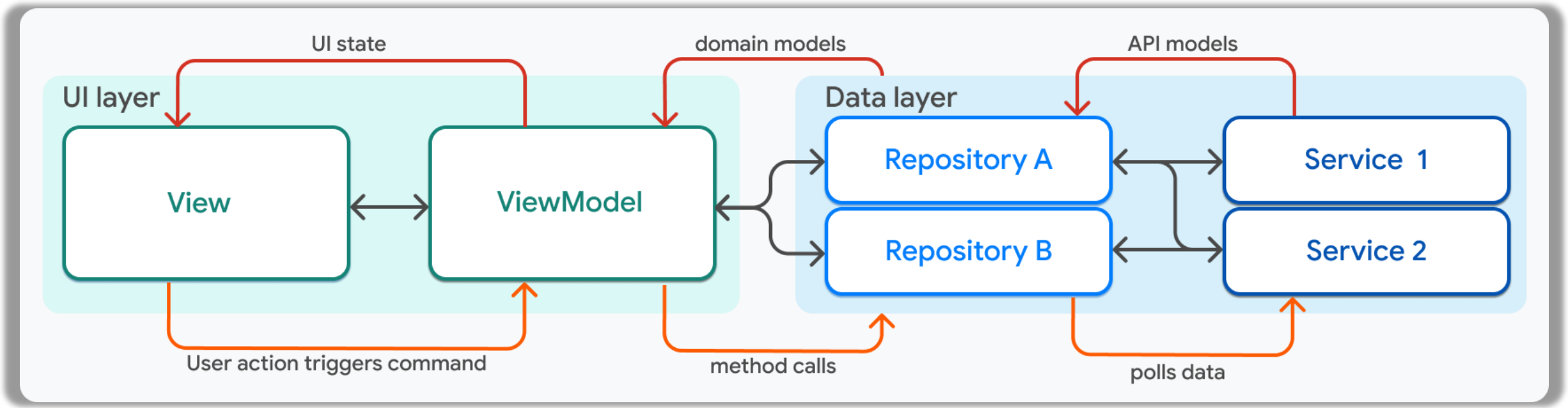
Data Model in App

```
Quote({required this.quote, required this.author, required this.categories});
// Factory constructor to create a Quote object from JSON
factory Quote.fromJson(Map<String, dynamic> json) {
  return Quote(
    quote: json['quote'] ?? '',
    author: json['author'] ?? '',
    categories: json['categories'] != null
      ? List<String>.from(json['categories'])
      : [],
    );
}
// Method to convert Quote object back to JSON
Map<String, dynamic> toJson() {
  return {'quote': quote, 'author': author, 'categories': categories};
}
```

Model-View-ViewModel
MVVM Architectural Pattern



MVVM



More details

MVVM

- Separation of concerns
- Testability
- Scalability

Quote App

1. Create Quote App flutter Project
2. Add http, flutter_riverpod and envied Packages
3. Add permission in **AndroidManifext.xml** in Android Project

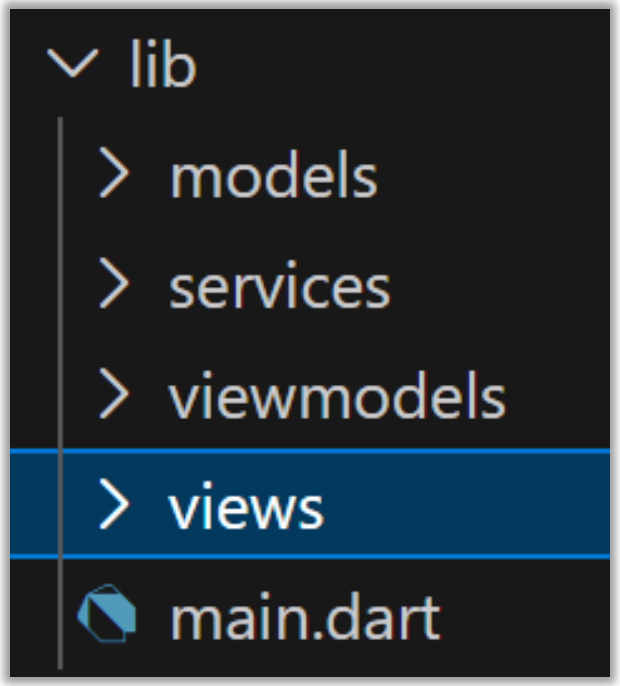
```
<!-- Required to fetch data from the internet. -->
<uses-permission android:name="android.permission.INTERNET" />
```

android\app\src\main\AndroidManifest.xml

Mac OS Settings

Quote App

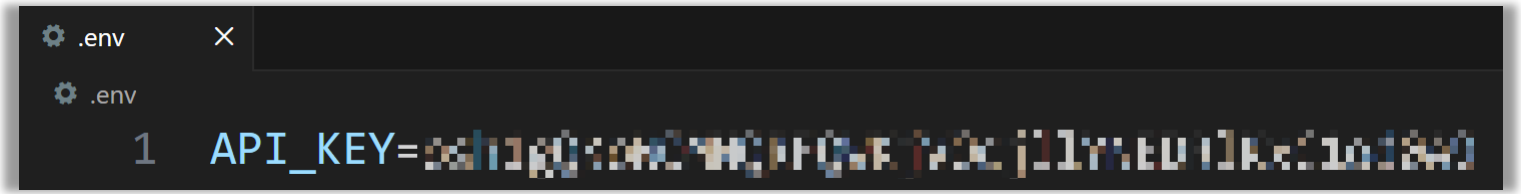
4. Folder Structure



Quote App

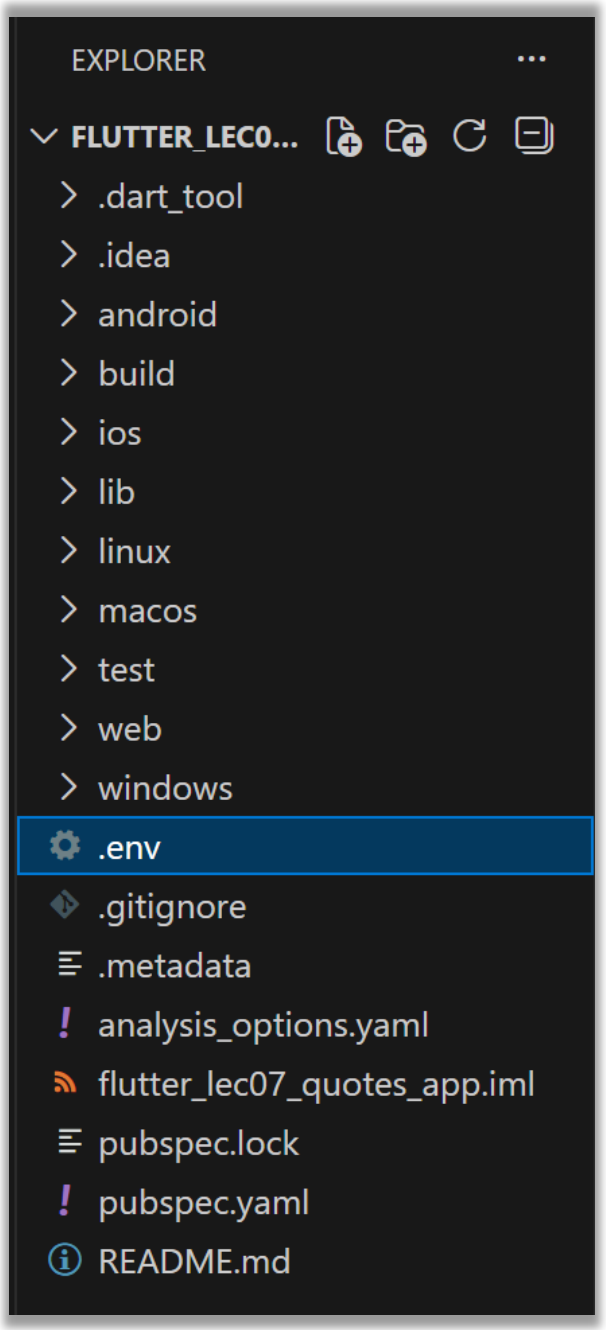
5. API KEY Management

- Create a .env file
- Save your API KEY



- Add dev dependencies

```
flutter pub add build_runner envied_generator --dev
```



Quote App

5. API KEY Management

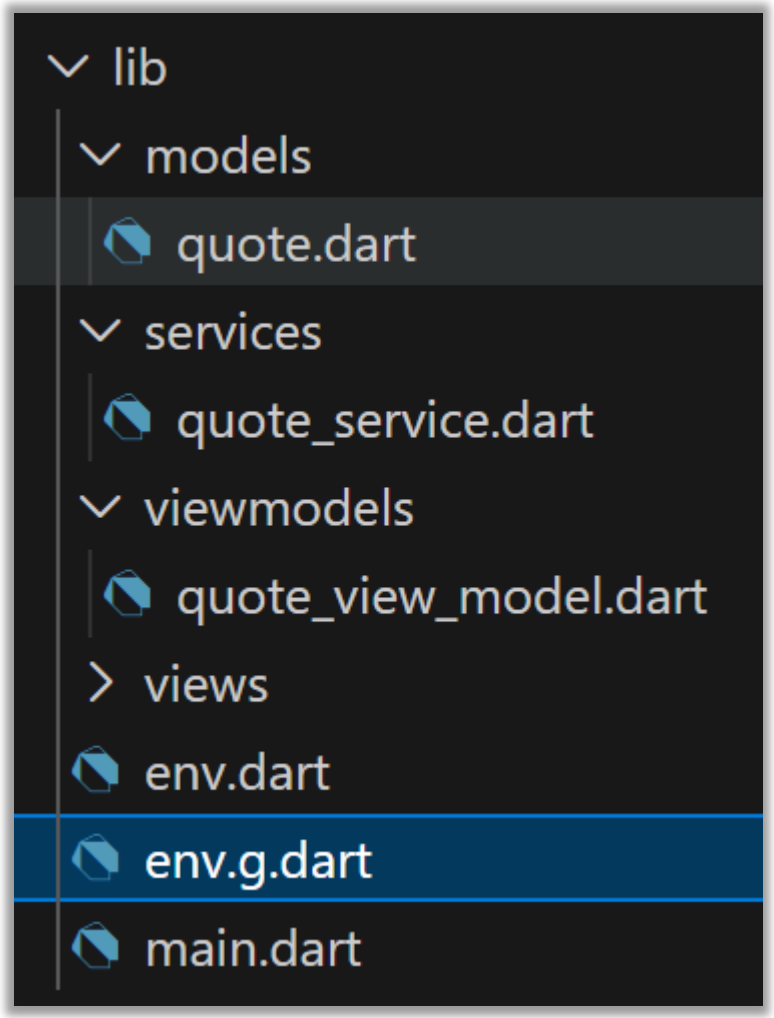
- Create an env.dart file in lib folder

```
import 'package:envied/envied.dart';
part 'env.g.dart';
@Envied(path: '.env')
abstract class Env {
  @EnviedField(varName: 'API_KEY', obfuscate: true)
  static final String apiKey = _Env.apiKey;
}
```

```
dart run build_runner build
```

Quote App

5. API KEY Management



Quote App

6. Create Model

```dart
class Quote {
  final String quote;
  final String author;
  final List<String> categories;
  Quote({required this.quote, required this.author, required this.categories});
  // Factory constructor to create a Quote object from JSON
  factory Quote.fromJson(Map<String, dynamic> json) {
    return Quote(
      quote: json['quote'] ?? '',
      author: json['author'] ?? '',
      categories: json['categories'] != null
          ? List<String>.from(json['categories'])
          : [],
    ); // Quote
  }
  // Method to convert Quote object back to JSON
  Map<String, dynamic> toJson() {
    return {'quote': quote, 'author': author, 'categories': categories};
  }
}
```

Quote App

7. Create API Service using HTTP Package

```dart
import 'package:http/http.dart' as http;

class QuoteService {
  final String baseUrl = 'https://api.api-ninjas.com';
```

Quote App - QuoteService

```dart
Future<List<Quote>> fetchQuoteOfDay() async {
  final String quoteUrl = '/v2/quoteoftheday';
  final response = await http.get(
    Uri.parse(baseUrl + quoteUrl),
    headers: {'X-Api-Key': Env.apiKey},
  );
  if (response.statusCode == 200) {
    List<dynamic> quotesListJson = jsonDecode(response.body);
    return quotesListJson
        .map((q) => Quote.fromJson(q as Map<String, dynamic>))
        .toList();
  } else {
    return [];
  }
}
```

Quote App - QuoteService

```dart
Future<List<Quote>> fetchQuotes(String categories, int qLimit) async {
  final String quoteUrl = '/v2/quotes?categories=$categories&limit=$qLimit';
  final response = await http.get(
    Uri.parse(baseUrl + quoteUrl),
    headers: {'X-Api-Key': Env.apiKey},
  );
  if (response.statusCode == 200) {
    List<dynamic> quotesListJson = jsonDecode(response.body);
    return quotesListJson
        .map((q) => Quote.fromJson(q as Map<String, dynamic>))
        .toList();
  } else {
    return [];
  }
}
```

Quote App - QuoteService

```
Future<List<Quote>> fetchRandomQuotes(String categories) async {
  final String quoteUrl = '/v2/randomquotes?categories=$categories';
  final response = await http.get(
    Uri.parse(baseUrl + quoteUrl),
    headers: {'X-Api-Key': Env.apiKey},
  );
  if (response.statusCode == 200) {
    List<dynamic> quotesListJson = jsonDecode(response.body);
    return quotesListJson
        .map((q) => Quote.fromJson(q as Map<String, dynamic>))
        .toList();
  } else {
    return [];
  }
}
```

Quote App

8. Create QuoteViewModel using flutter_riverpod

```
class QuoteViewModel extends AsyncNotifier<List<Quote>> {
  final QuoteService apiService = QuoteService();
  @override
  FutureOr<List<Quote>> build() {
    return _loadQuoteOfDay();
  }
  Future<List<Quote>> _loadQuoteOfDay() async {
    return await apiService.fetchQuoteOfDay();
  }
```

Quote App - QuoteViewModel

```
Future<void> loadQuotes({
  String categories = 'wisdom',
  int qLimit = 1,
}) async {
  state = const AsyncLoading();
  try {
    final updated = await apiService.fetchQuotes(categories, qLimit);
    state = AsyncData(updated);
  } catch (e, st) {
    state = AsyncError(e, st);
  }
}
```

Quote App - QuoteViewModel

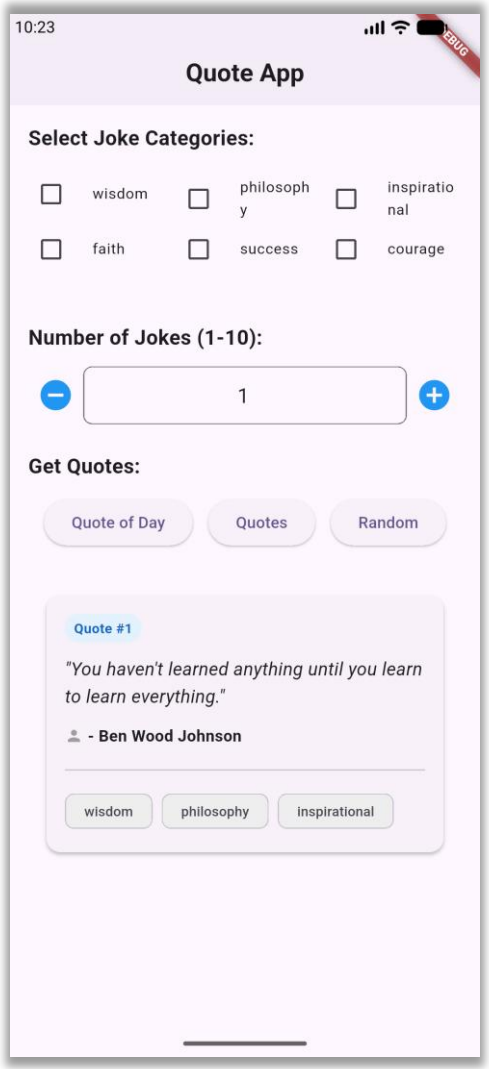
```
Future<void> loadQuoteOfDay() async {
  state = const AsyncLoading();
  try {
    final updated = await apiService.fetchQuoteOfDay();
    state = AsyncData(updated);
  } catch (e, st) {
    state = AsyncError(e, st);
  }
}
```

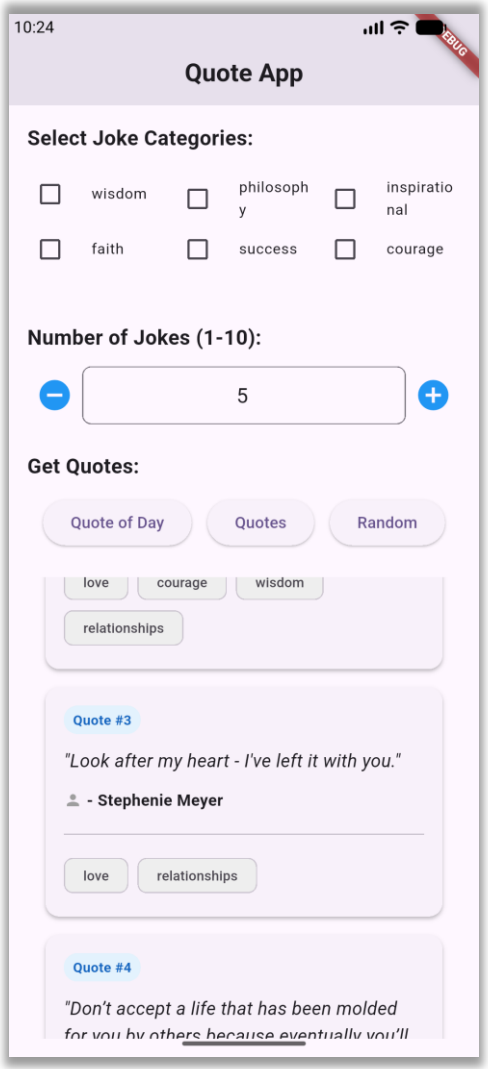
Quote App - QuoteViewModel

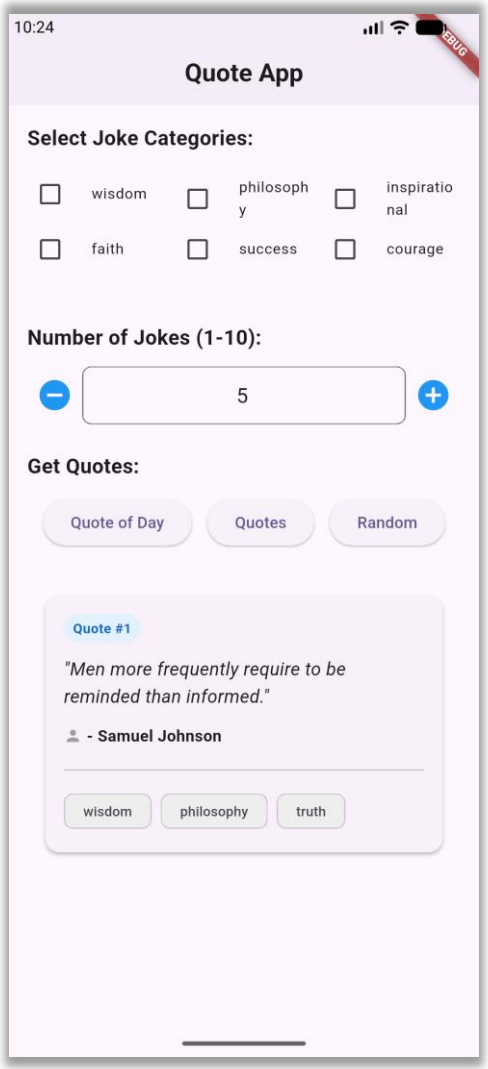
```
Future<void> loadRandomQuotes({String categories = 'wisdom'}) async {
  state = const AsyncLoading();
  try {
    final updated = await apiService.fetchRandomQuotes(categories);
    state = AsyncData(updated);
  } catch (e, st) {
    state = AsyncError(e, st);
  }
}
```

```
final quoteViewModel =
AsyncNotifierProvider<QuoteViewModel, List<Quote>>(
  QuoteViewModel.new,
);
```

Quote App - GUI







References

- https://docs.flutter.dev/data-and-backend/networking
- https://pub.dev/packages/http
- https://www.restapitutorial.com/introduction
- https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status
- https://api-ninjas.com/api/quotes
- https://docs.flutter.dev/app-architecture/guide